Coverage-Guided LLM-Aided Testbench Generation
Mitsuki Nakajima

1. Pipeline Overview

This project extends an LLM-aided testbench generation workflow with coverage-guided feedback. Instead of relying only on an LLM to generate a fixed set of test patterns, the generated patterns are first simulated using Icarus Verilog and analyzed using Covered. Uncovered RTL behaviors are then extracted from the coverage reports and provided back to the LLM so that it can generate additional targeted test patterns.

The overall workflow used in this project was: RTL and natural-language specification → initial LLM-generated patterns → stimulus-only Verilog testbench → Icarus Verilog simulation → Covered coverage analysis → extraction of uncovered targets → LLM-generated targeted patterns → repeated coverage analysis → coverage-optimized pattern set → Python golden reference model → expected outputs → final self-checking Verilog testbench.

Coverage optimization was performed before golden-output generation because coverage and functional correctness answer different questions. Coverage determines whether the generated stimuli exercise the RTL structure, while the Python golden model determines whether the DUT outputs are correct for those stimuli.

2. Icarus Verilog and Covered Coverage Demo

Before integrating coverage analysis into the LLM pipeline, I reproduced the provided `simple_alu` coverage example.

The original testbench intentionally exercised ADD and AND while leaving OR and XOR insufficiently tested. The baseline coverage was:

Metric	**Baseline**
Line coverage	60%
Toggle 0 → 1	64%
Toggle 1 → 0	21%
Combinational logic	35%

I then added targeted OR and XOR stimuli and reran the Icarus Verilog and Covered flow. After adding targeted OR and XOR stimuli, the coverage improved to:

Metric	Baseline	After OR/XOR
Line coverage	60%	100%
Toggle 0 → 1	64%	71%
Toggle 1 → 0	21%	36%
Combinational logic	35%	53%

This demonstrated that adding targeted stimuli based on missing operations improved structural coverage, especially line and combinational-logic coverage.

One compatibility issue occurred between the VCD produced by the installed Icarus Verilog version and Covered 0.7.10. A VCD comment beginning with $comment Show was removed before coverage scoring so that Covered could parse the waveform successfully.

3. Original LLM Pipeline Warm-Up
For the original LLM-only workflow, I used the 2-to-1 multiplexer tutorial example. The LLM generated the stimulus patterns and the Python golden reference model. The expected outputs were then added to the generated test patterns and used to create a self-checking Verilog testbench.

The final testbench compiled successfully using Icarus Verilog and completed with the expected PASS results. This warm-up confirmed that the original pattern-generation, golden-model, and self-checking-testbench pipeline worked before coverage feedback was integrated.

4. 8-Bit ALU Baseline
The main experiment used the provided 8-bit multi-function ALU. The DUT implements ADD, SUB, AND, OR, XOR, NOT, logical left shift, logical right shift, arithmetic right shift, signed less-than comparison, and default behavior for unsupported opcodes.

The initial test set was generated by the LLM without giving it any coverage information. The LLM produced 16 initial patterns.

The baseline coverage results were:

Metric	**Iteration 0**
Number of patterns	16
Line coverage	100%
Toggle 0 → 1	100%
Toggle 1 → 0	97%
Combinational logic	63%

Although line and toggle coverage were already high, combinational-logic coverage was substantially lower. The detailed report showed missing combinations in arithmetic, overflow-detection, bitwise, and shift expressions. These uncovered behaviors became the feedback targets for the refinement loop.

5. Coverage-Guided Pattern Refinement

For each refinement iteration, the Covered summary and detailed reports were analyzed. Coverage metrics and uncovered logic combinations were converted into a compact LLM prompt. The LLM was instructed to generate additional patterns only, while retaining all existing patterns.

Iteration 1

The first refinement added 12 patterns, producing a total of 28.

Coverage changed from: Logic: 63% → 71%, Toggle 1→0: 97% → 100%. Line coverage and toggle 0→1 coverage remained at 100%. The new patterns targeted remaining ADD/SUB combinations, overflow logic, AND/OR/XOR states, and shift-operation states.

Iteration 2

The second refinement added another 12 patterns, producing a total of 40.

Combinational-logic coverage increased substantially from 71% to 90%, while all line and toggle metrics remained at 100%. The additional patterns concentrated on the specific ADD, SUB, XOR, and shift-expression states still listed as uncovered by Covered.

Iteration 3

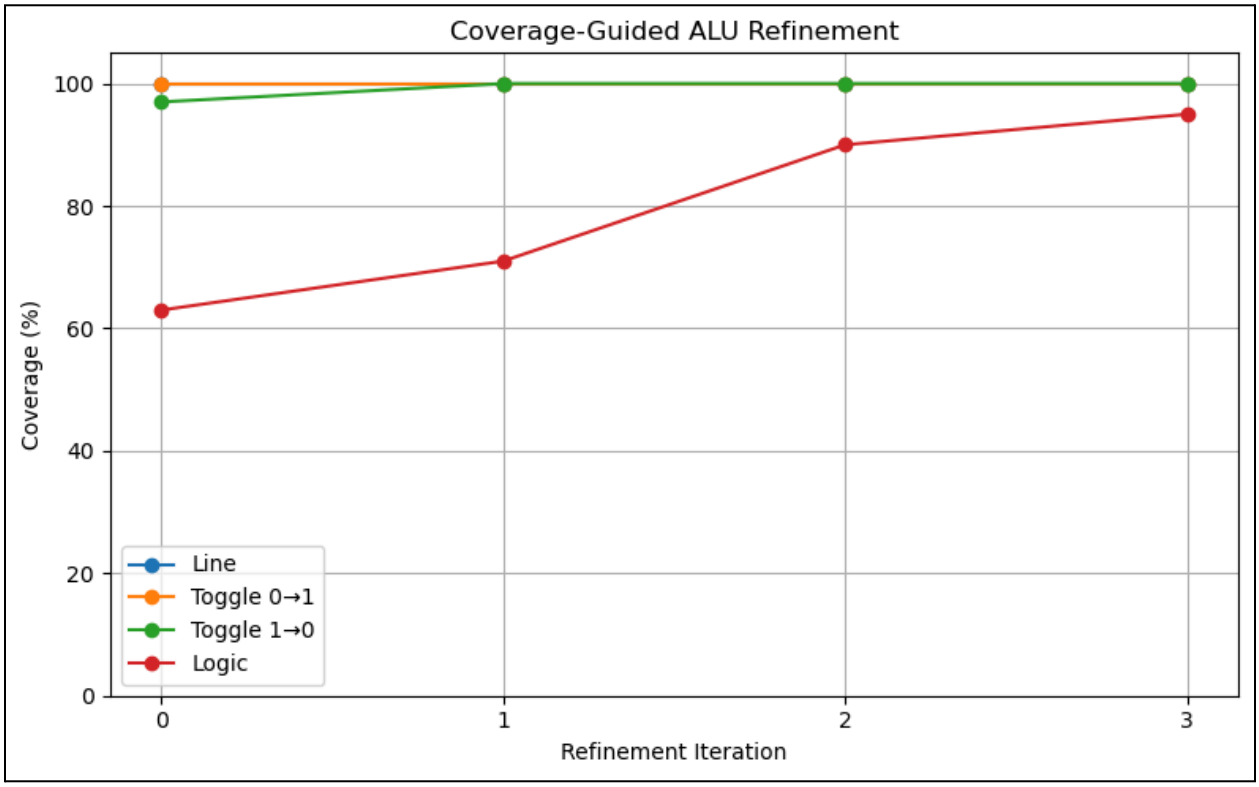
The final refinement added 9 patterns, producing a total of 49 patterns. This iteration also explicitly preserved an explanation mapping each LLM-generated pattern to the uncovered behavior it was intended to exercise.

Final coverage was:

Metric	Iteration 0	Iteration 1	Iteration 2	Iteration 3
Patterns	16	28	40	49
Line	100%	100%	100%	100%
Toggle 0 → 1	100%	100%	100%	100%
Toggle 1 → 0	97%	100%	100%	100%
Logic	63%	71%	90%	95%

The final logic coverage reached 95%, or 60 of 63 logic combinations.

The refinement process was stopped at this point because line and toggle coverage were complete and combinational-logic coverage had improved substantially from 63% to 95%. The remaining uncovered combinations were limited to a small number of ADD/SUB expression states, so further iterations were not necessary to demonstrate the effectiveness of the coverage-guided workflow.



6. Golden Reference Model and Final Self-Checking Testbench

After coverage refinement was complete, the final 49-pattern optimized test set was passed to a Python golden reference model. The golden model implements the ALU specification

independently in Python and calculates the expected values of: `y, zero, negative, carry, overflow.` The expected outputs were stored together with each stimulus pattern in `alu_test_patterns_with_golden.json`. A final self-checking Verilog testbench, `alu_testbench_final.v`, compared every DUT output against the corresponding golden output.

The correct ALU compiled successfully and the final simulation reported:

```
TOTAL FAILURES: 0
```

Therefore, all 49 coverage-optimized test patterns produced outputs matching the independent Python reference model.

7. Intentional Bug-Detection Experiment

To evaluate whether coverage-guided refinement improved functional bug detection, I introduced a small intentional bug into a copy of the ALU.

The correct arithmetic-right-shift implementation was:

```
y = $signed(a) >>> b[2:0];
```

The buggy version used logical right shift instead:

```
y = a >> b[2:0];
```

This causes incorrect behavior when the input operand is negative, because arithmetic right shift should replicate the sign bit while logical right shift inserts zeros.

Baseline Pattern Set

The original 16-pattern LLM-only test set detected one failure:

```
a = 128, b = 7, op = 8
```

The buggy DUT produced `y = 1`, while the golden model expected `y = 255.`

Result:

```
TOTAL FAILURES: 1
```

Coverage-Optimized Pattern Set

The final 49-pattern set detected two failing cases.

The first was the same case detected by the baseline:

```
a = 128, b = 7, op = 8
```

The optimized set also detected another arithmetic-right-shift failure:

```
a = 128, b = 1, op = 8
```

For this second case, the buggy DUT produced `y = 64`, while the golden result was `y = 192`.

Result:

```
TOTAL FAILURES: 2
```

Therefore, the baseline test set was already capable of exposing this particular bug, but coverage-guided refinement increased the number of distinct stimuli that exposed the defect. This indicates that the optimized pattern set exercised the SRA behavior more thoroughly and provided stronger evidence of the implementation error.

After the bug experiment, the original correct ALU was restored and tested with all 49 optimized patterns. The final self-checking simulation completed with:

```
TOTAL FAILURES: 0
```

8. Limitations

Higher structural coverage does not guarantee complete functional verification.

For example, the baseline pattern set in this experiment already detected the intentionally introduced SRA bug even though its combinational-logic coverage was only 63%. Conversely, a test suite could potentially reach very high line, toggle, and logic coverage while still failing to test a particular semantic relationship between inputs and outputs.

Coverage should therefore be treated as a stimulus-quality signal rather than a correctness oracle. Covered identifies which RTL structures and logic combinations were exercised, while the independent Python golden model determines whether the resulting behavior is functionally correct.

Another limitation is that LLM-generated targeted patterns are dependent on the quality of the coverage feedback supplied in the prompt. Some generated patterns may duplicate existing tests or fail to hit the exact intended coverage state. Simulation and repeated coverage analysis are therefore still required to validate every refinement.

9. Conclusion

The coverage-guided workflow substantially improved the structural quality of the automatically generated ALU tests.

Starting from 16 LLM-generated patterns, combinational-logic coverage increased from 63% to 95% after three refinement iterations, while line and toggle coverage reached 100%. The final optimized set contained 49 patterns.

The optimized patterns were then labeled using an independent Python golden model and verified using a self-checking Verilog testbench. The correct DUT passed all 49 tests with zero failures.

In the intentional SRA-to-SRL bug experiment, the baseline set detected one failing case, while the coverage-optimized set detected two. This demonstrates that coverage-guided refinement can increase the diversity and effectiveness of generated test stimuli, even when high structural coverage alone cannot guarantee complete functional correctness.